

Skin Slide

Arun Somasundaram
arun.somasundaram-
@dreamworks.com
DreamWorks Animation
USA

Pramita Mukherjee
pramita.mukherjee-
@dreamworks.com
DreamWorks Animation
USA

Jinnah Yu
jinnah.yu@dreamworks.com
DreamWorks Animation
USA

Ron Williams
ron.williams@dreamworks.com
DreamWorks Animation
USA



Figure 1: *Skin Slide* was used on the bears from the DreamWorks film *Puss in Boots: The Last Wish*

ABSTRACT

Skin Slide, a tool developed at DreamWorks Animation, slides and jiggles skin along the original animated surface solely based on the input animated skin and artist-controlled parameters. This effect brings across the weight of characters on screen making the motion more believable. In addition, the predominant sliding motion created by this tool does not alter the character's silhouette - a feature desired by animators. *Skin Slide* is artist-friendly by being robust, fast, intuitive, and easily controllable for setup and shot work.

ACM Reference Format:

Arun Somasundaram, Pramita Mukherjee, Jinnah Yu, and Ron Williams. 2023. Skin Slide. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH '23 Talks)*, August 06–10, 2023, Los Angeles, CA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3587421.3595431>

1 MOTIVATION

Using real life as a reference, the skin movement provides the majority of the secondary motion on furry creatures, especially the heavy creatures, such as the bears [Figure 1], with fur riding on the skin. While adding secondary motion to the animated skin, the artistic direction on the film required that the original silhouette of the skin not be affected much and that the resulting skin be free of new wrinkles - to keep it clean, simple, and avoid distracting crisscrossing artifacts with the fur that is riding on top. The process of adding secondary motion to skin needed to work with non-physical exaggerated animation as can be seen in animated films.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH '23 Talks, August 06–10, 2023, Los Angeles, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0143-6/23/08.

<https://doi.org/10.1145/3587421.3595431>

A physically realistic skin simulation model is complex and involves the construction and simulation of anatomical layers that drive the skin such as bones, muscles, fascia, and fat. It requires the upfront work of multiple departments including *Modeling*, *Rigging*, *Animation*, and *Character Effects*. Such a simulation model is time intensive, heavy, and challenging to artistically control. It will also have difficulties maintaining the original silhouette, avoiding wrinkles, and working with non-physical exaggerated animation.

The *Skin Slide* tool addresses all the above issues while providing sliding skin jiggle motion just from the animated skin and artist parameters. It allows the skin to slide on the original animated surface without changing silhouette or adding new wrinkles. Both setup and shot work can be done by the *Character Effects* department, thereby minimizing cost. In addition, it is efficient and intuitive, robust on exaggerated animation, and provides enough control (including local control) to achieve the required artistic direction.

2 SKIN SLIDE

The predominant motion for *Skin Slide* is jiggle motion that slides on the original animated skin surface. However, the tool also allows for jiggle motion to lie outside the original skin surface. The steps in this process are described next.

- **Pre-Processing:** The input to *Skin Slide* is an animated mesh, $mesh_{anim}$. The simulation can be turned off in unneeded regions by setting the envelope point attribute to zero. $mesh_{anim}$ is then triangulated. The original mesh is wrapped to the skin-slid triangulated mesh for the final output. The simulated mesh, $mesh_{sim}$, is initialized to be a copy of the mesh $mesh_{anim}$ at the first frame.
- **Simulation:** Springs are created between corresponding vertices of $mesh_{anim}$ and $mesh_{sim}$ to create a mass-spring system. A mass-spring simulation is performed as the character is animated, causing $mesh_{sim}$ to be pulled by $mesh_{anim}$. Simulation parameters such as mass, spring constant, spring damping, global

damping, and substeps can be controlled by the artist and locally scaled by setting point attributes (typically done by skin painting). The simulation is multithreaded across vertices for speed.

- **Orthogonal Motion Removal:** Let $frame_{cur}$ be the time frame for which the simulation is evaluated. For sliding-only motion, first, motion that is orthogonal/normal to $mesh_{anim}$ at $frame_{cur}$ is removed, keeping only tangential motion. Though this process helps with the sliding motion, maintaining tangential motion alone does not ensure that the resulting vertex lies on the surface of the original skin. For non-sliding motion, the artist is given control (including local control) to remove any fraction of the normal motion as needed. $mesh_{sim}$ is the result of normal motion removal at $frame_{cur}$ after this step. For each vertex, let dir_{sim_init} be the difference between corresponding vertex positions of $mesh_{sim}$ and $mesh_{anim}$, namely, $dir_{sim_init} = pos_{vert_mesh_sim} - pos_{vert_mesh_anim}$ at $frame_{cur}$. dir_{sim_init} specifies the direction of simulation of the vertex $vert_{anim}$ (of $mesh_{anim}$) at $frame_{cur}$.

- **Sliding Walk:** Next, for sliding-only motion, a crucial walking step is performed that ensures that the resulting $mesh_{sim}$ lies on the surface of $mesh_{anim}$ at $frame_{cur}$. Each vertex, $vert_{anim}$, is walked on $mesh_{anim}$ in the direction indicated by dir_{sim_init} . The first walking step of a vertex starts from that vertex and is on the neighboring triangle, tri_{chosen} , sharing that vertex. When a boundary of a triangle is hit, the walking process continues onto an adjacent triangle, $tri_{adjacent}$. The walking process iteratively continues until the length walked is $len_{dir_sim_init}$, where $len_{dir_sim_init}$ is the length of dir_{sim_init} .

The walking process is illustrated with the help of Figure 2. During the first walking step, let $dir_{sim} = dir_{sim_init}$ and $len_{dir_sim} = len_{dir_sim_init}$. For a vertex $vert_{anim}$, with position pos_{vert} , its corresponding vector dir_{sim} is projected (while maintaining its length) onto the planes, $planes_{proj}$, of all the triangles, $tris_{vert}$, sharing that vertex. One of the projected vectors, dir_{sim_proj} , will intersect a triangle's, tri_{chosen} 's, edge segment, $edge_{intersect}$, which is opposite to $vert_{anim}$. That is the triangle on which $vert_{anim}$ takes its first walk. Let the intersection point be $point_{intersect}$ with position $pos_{intersect}$. Let $dir_{intersect}$ be the first walking direction, where $dir_{intersect} = pos_{intersect} - pos_{vert}$. Let $len_{intersect}$ be the length of $dir_{intersect}$. If $(len_{dir_sim} \leq len_{intersect})$, then the final walked position of $vert_{anim}$ is pos_{walk} , where $pos_{walk} = pos_{vert} + \widehat{dir_{intersect}} * len_{dir_sim}$. If $(len_{dir_sim} > len_{intersect})$, then $vert_{anim}$ is moved to $pos_{intersect}$ ($pos_{vert} = pos_{intersect}$), dir_{sim_proj} is reduced in length by $len_{intersect}$, and the walking process continues with dir_{sim_proj} being the new dir_{sim} , and $len_{dir_sim} = len_{dir_sim} - len_{intersect}$, in the next walking step. This walking process iteratively continues until a distance of $len_{dir_sim_init}$ is walked. It should be noted that during a walking step, if the intersection point $point_{intersect}$ happens to be on one of the corners of $edge_{intersect}$ (i.e., a vertex on that edge), a projection of dir_{sim} onto the neighboring triangles' planes (similar to the first walking step) needs to be taken to identify the triangle for the next walking step. If the intersection happens to be internal to $edge_{intersect}$, the next walking step is on the adjacent triangle, $tri_{adjacent}$, sharing $edge_{intersect}$.

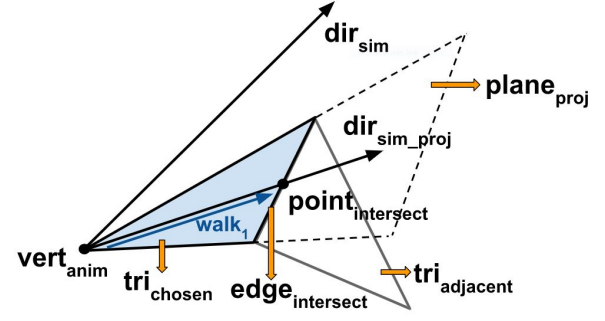


Figure 2: The first walk, $walk_1$, of vertex $vert_{anim}$ on the triangle tri_{chosen} is illustrated in this figure. The second walk starting from $point_{intersect}$ will happen on $tri_{adjacent}$.

The artist has three options to modify the length of the walk. The artist can set the point attribute $walk_{dist_scale}$ to scale $len_{dir_sim_init}$ locally, and/or set $walk_{dist_max}$ point attribute to restrict that length locally. The artist can also restrict the walk length at a walk step based on the direction of traversal relative to dir_{sim_init} . So, at any walk step, $len_{dir_sim} = \max(0, \dot{dir_{sim_init}} \cdot \widehat{dir_{sim}}) * len_{dir_sim}$. After the final walking step, we obtain the final skin-slid mesh $mesh_{walk}$ for $frame_{cur}$. This $mesh_{walk}$ becomes the initial state for the next frame's mass-spring simulation, which gets pulled by the deforming $mesh_{anim}$.

This walking process is robust, controllable, and fast. The walk is based on topologically connectivity and is not derailed by self-intersecting meshes or exaggerated animation. The scaling controls allow for sliding while avoiding vertices pinching or crisscrossing during the walk. After the initial mass-spring simulation, the artist can easily modulate the slide quickly with local scaling controls. In addition, the process allows for parallel threading across vertices for increased speed.

3 IMPLEMENTATION

The *Skin Slide* tool is implemented using custom nodes in a third-party package and is multithreaded to run in parallel across vertices. The tool provides various parameters for the mass-spring simulation, orthogonal motion removal, and the walking step. In addition, the tool allows for creating both sliding and non-sliding motion, and the ability to locally blend between them. Local control can be achieved by setting scaling point attributes on the skin mesh.

4 RESULTS AND CONCLUSION

The *Skin Slide* tool was used on the film *Puss in Boots: The Last Wish* on the bears. This unique approach has been tested on hundreds of shots and modified to fit artistic direction, and maintain control, speed, and robustness. Since the process works off just the animated skin mesh, the setup cost across departments is minimal. All *Character Effects* artists on the film used *Skin Slide*. It allowed the film's artistic direction for wrinkle-free secondary motion that preserved the original silhouette to be achieved quickly without a complex bone-muscle-fascia-fat-skin system. *Skin Slide* enhanced the motion of the heavy characters making them more believable.